Razvoj softvera I – vježbe 2

Prvi Web API korak po korak (MojaApp) · akademska 2026/27.

Pravimo mali Web API za evidenciju studenata. Podaci se prvo čuvaju u memoriji, a u tački 5 ih prebacujemo u bazu na SQL Serveru. Uz svaki korak je kompletan kod: fajl možete prepisati ili kopirati u Visual Studio. Pročitajte i objašnjenje ispod koda, jer na istim pojmovima počiva starter projekat na kojem radite seminarski rad. Gotov projekat je u folderu V01, u „V01 - MojaApp početni kod.zip".

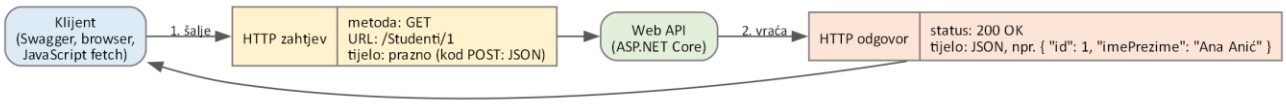
1. Od fetch-a do akcije

Na prvoj godini ste iz JavaScripta pozivali gotov API:

```
// JavaScript klijent, kao na prvoj godini
const odgovor = await fetch("http://localhost:5183/Studenti/1");
const student = await odgovor.json();
console.log(student.imePrezime);
```

Sada pravimo drugu stranu. Na serveru ovaj zahtjev obrađuje jedna metoda u C# klasi: klasa je **kontroler**, a metoda je **akcija**.

Svaki HTTP razgovor ima dva dijela: zahtjev koji šalje klijent i odgovor koji vraća API.

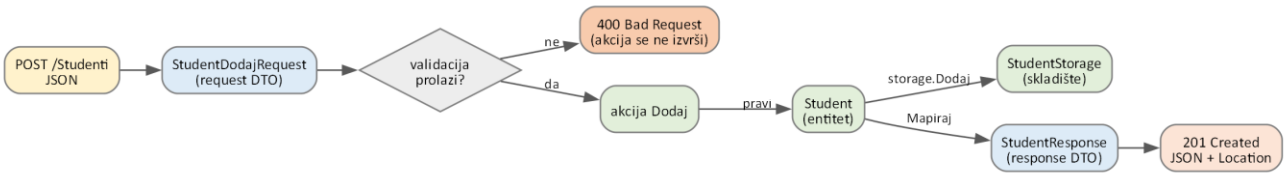


Zahtjev klijenta	Akcija u kontroleru	Odgovor
`GET /Studenti`	`GetAll()`	200 i lista studenata
`GET /Studenti/1`	`GetById(1)`	200 i student, ili 404
`POST /Studenti` + JSON u tijelu	`Dodaj(request)`	201 i novi student, ili 400

Statusni kodovi koje ćemo koristiti: **200** OK, **201** Created (nešto je napravljeno), **204** No Content (uspjelo, nema tijela), **400** Bad Request (neispravni podaci), **404** Not Found, **409** Conflict (npr. duplikat) i **500** (greška na serveru, koju klijent ne bi trebao vidjeti).

2. Pojmovi

Put podataka kroz API, na primjeru dodavanja studenta (POST):

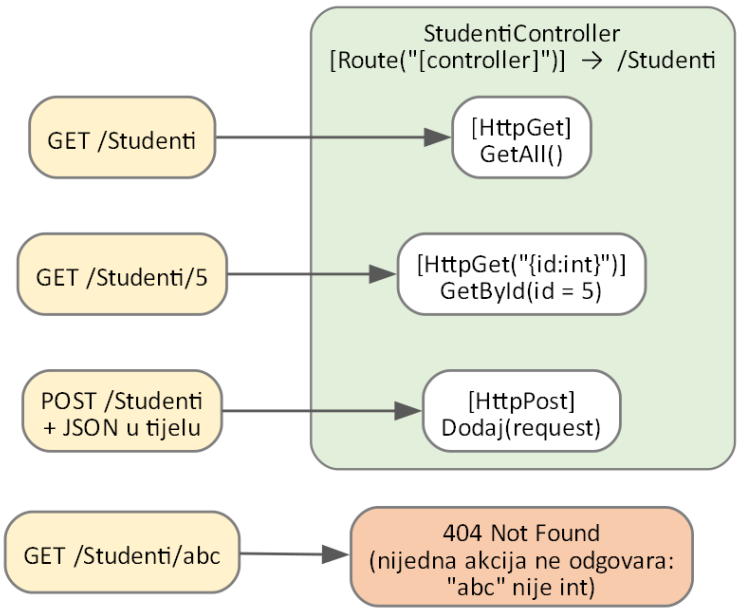


- **Kontroler** je klasa koja grupiše akcije za jedan resurs, ovdje studente. Nasljeđuje `ControllerBase`. Atribut `[ApiController]` uključuje automatsku validaciju i standardni format greške. `[Route("[controller]")]` znači da je ruta naziv klase bez riječi Controller: `StudentiController` → `/Studenti`.
- **Akcija** je javna (`public`) metoda u kontroleru. HTTP atribut (`[HttpGet]`, `[HttpPost]`, `[HttpPut]`, `[HttpDelete]`) određuje koju HTTP metodu obrađuje, a dodatak rute, npr. `"{id:int}"`, čita dio URL-a.

Parametri akcije se popunjavaju sami: iz rute (`id`), iz query stringa (`?pretraga=ana`) ili iz tijela zahtjeva (JSON → objekat). Ono što akcija vrati, ASP.NET pretvara u JSON.

- **Entitet** (entity klasa) opisuje podatak onako kako ga aplikacija čuva, ovdje `Student`. Kad dodamo bazu, svaki entitet postaje tabela. Entitet ima i polja koja klijent ne smije mijenjati, kao što su `Id` i `DatumKreiranja`.
- **DTO** (Data Transfer Object) opisuje šta putuje preko mreže. **Request DTO** kaže šta klijent smije poslati, a **response DTO** šta dobija nazad.
- **Validacija** se piše atributima na request DTO-u (`[Required]`, `[StringLength]`, `[RegularExpression]`). `[ApiController]` provjeri podatke prije akcije. Ako nisu ispravni, vraća 400 s opisom greške, a akcija se uopšte ne izvrši.
- **Dependency Injection (DI):** kontroler ne pravi skladište sam (`new StudentStorage()`), nego ga dobija kroz konstruktor. U `Program.cs` se registruje koji servisi postoje i koliko dugo žive.
- **Swagger** već znate: stranica s opisom API-ja na kojoj se zahtjevi mogu isprobati. ASP.NET je pravi sam, iz kontrolera, akcija i DTO-ova.

Kako zahtjev nađe svoju akciju:



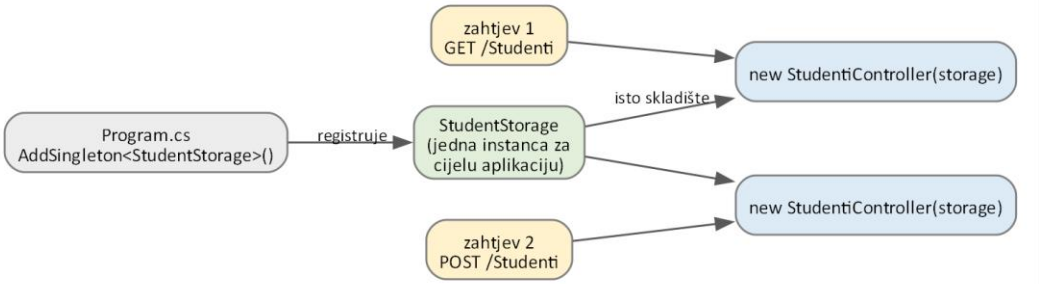
Zašto DTO, a ne entitet direktno?

- Klijent ne može podmetnuti `Id` ili `DatumKreiranja`, jer ih request DTO nema.
- Odgovor sadrži samo ono što klijentu treba, u obliku koji mu treba (npr. `ImePrezime` umjesto dva polja).
- Entitet se može mijenjati, a API za klijenta ostaje isti.
- U starter projektu i u seminarskom radu entitet se nikad ne vraća direktno kroz API.

Vijek trajanja servisa u DI:

Registracija	Vijek trajanja	Gdje se koristi
`AddSingleton`	jedna instanca za cijelu aplikaciju	naše skladište u memoriji
`AddScoped`	nova instanca za svaki HTTP zahtjev	`DbContext` (baza) u starter projektu
`AddTransient`	nova instanca svaki put kad se zatraži	mali servisi bez stanja

Skladište je singleton: svaki zahtjev dobija novi kontroler, ali svi kontroleri dijele isto skladište.



3. Korak po korak

3.1 Novi projekat

1. Visual Studio → Create a new project → **ASP.NET Core Web API**:
   - Project name `MojaApp.API`, Solution name `MojaApp`, lokacija npr. `C:\rs1\mojaapp`;
   - Framework **.NET 8.0 (Long Term Support)**, Authentication type None;
   - uključeno „Use controllers" i „Enable OpenAPI support".
   - Ili iz komandne linije:

```
dotnet new sln -n MojaApp
dotnet new webapi --use-controllers -f net8.0 -n MojaApp.API -o MojaApp.API
dotnet sln add MojaApp.API/MojaApp.API.csproj
```

Šta ste dobili:

Fajl	Uloga
`Program.cs`	Početna tačka: registracija servisa i pokretanje aplikacije.
`Controllers/WeatherForecastController.cs`	Primjer kontrolera s jednom akcijom.
`WeatherForecast.cs`	Primjer klase koju akcija vraća.
`Properties/launchSettings.json`	Profili pokretanja i portovi (npr. `http://localhost:5183`). Port je kod svakog drugačiji.
`appsettings.json`	Konfiguracija (kasnije connection string za bazu).
`MojaApp.API.http`	Fajl za slanje HTTP zahtjeva direktno iz Visual Studija.

3.2 Pokrenite primjer, pa ga obrišite

1. Pokrenite projekat (F5). Otvara se Swagger.
2. Otvorite `GET /WeatherForecast` → Try it out → Execute. Pogledajte JSON odgovor i status 200.
3. Otvorite `WeatherForecastController.cs` i nađite ono što smo opisali u tački 2: `[ApiController]`, `[Route]`, `[HttpGet]` i povratnu vrijednost akcije.
4. Zaustavite aplikaciju i obrišite `WeatherForecast.cs` i `Controllers/WeatherForecastController.cs`.

3.3 Entitet Student

Napravite folder `Models` i u njemu klasu:

Models/Student.cs

```
namespace MojaApp.API.Models;

public class Student
{
    public int Id { get; set; }
    public string Ime { get; set; } = "";
    public string Prezime { get; set; } = "";
    public string BrojIndeksa { get; set; } = "";
    public DateOnly? DatumRodjenja { get; set; }
    public DateTime DatumKreiranja { get; set; }
}
```

- `= ""` daje početnu vrijednost: string tako nikad nije `null`.
- `DateOnly?` je datum bez vremena. Znak `?` znači da polje može ostati prazno.
- `Id` i `DatumKreiranja` postavlja server, ne klijent.

3.4 Skladište u memoriji

Napravite folder `Data` i u njemu klasu:

Data/StudentStorage.cs

```
using MojaApp.API.Models;

namespace MojaApp.API.Data;

// Privremeno skladište u memoriji: podaci nestaju kad se aplikacija ugasi.
// Registrovano kao singleton u Program.cs. Na narednim vježbama ga zamjenjuje baza (EF Core).
public class StudentStorage
{
    private readonly List<Student> _studenti = [];
    private int _sljedeciId = 1;

    public StudentStorage()
    {
        Dodaj(new Student { Ime = "Ana", Prezime = "Anić", BrojIndeksa = "IB260001", DatumRodjenja = new
DateOnly(2006, 3, 14) });
        Dodaj(new Student { Ime = "Edin", Prezime = "Edić", BrojIndeksa = "IB260002" });
    }

    public List<Student> GetAll() => _studenti;

    public Student? GetById(int id) => _studenti.FirstOrDefault(x => x.Id == id);

    public Student Dodaj(Student student)
    {
        student.Id = _sljedeciId++;
        student.DatumKreiranja = DateTime.UtcNow;
        _studenti.Add(student);
        return student;
    }
}
```

- Lista `_studenti` je `private`: drugi dijelovi aplikacije do studenata dolaze samo preko metoda.
- `_sljedeciId` dodjeljuje Id redom, kao što to kasnije radi baza.
- Konstruktor ubacuje dva studenta, da imate podatke za testiranje.
- `GetById` vraća `null` kad student ne postoji, zato je povratni tip `Student?`.
- Sitnice iz C#-a: `[]` je prazna lista, `=>` je metoda u jednom redu, a `FirstOrDefault` (LINQ) vraća prvi element koji odgovara uslovu ili `null`.

3.5 Registracija skladišta u Program.cs

Cijeli `Program.cs` treba izgledati ovako. Dodali smo `using` u prvom redu i jedan `AddSingleton`:

Program.cs

```
using MojaApp.API.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Jedna instanca za cijelu aplikaciju, inače bi svaki request dobio praznu listu.
builder.Services.AddSingleton<StudentStorage>();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

- Prvi dio (`builder.Services…`) kaže koji servisi postoje. Drugi dio (`app.Use…`, `app.Map…`) određuje kroz šta prolazi svaki zahtjev: Swagger, HTTPS i kontrolere.
- `AddSingleton<StudentStorage>()`: cijela aplikacija dijeli jedno skladište. Kad biste napisali `AddScoped`, svaki zahtjev bi dobio novo, prazno skladište: POST bi dodao studenta, a sljedeći GET ga ne bi vidio. Probajte.

3.6 Response DTO i prva akcija

Napravite folder `Dtos` i u njemu klasu za odgovor:

Dtos/StudentResponse.cs

```
namespace MojaApp.API.Dtos;

// Ono što server vraća. Klijent ne dobija entitet direktno.
public class StudentResponse
{
    public int Id { get; set; }
    public string ImePrezime { get; set; } = "";
    public string BrojIndeksa { get; set; } = "";
    public DateOnly? DatumRodjenja { get; set; }
}
```

U folderu `Controllers` napravite kontroler s prvom akcijom:

Controllers/StudentiController.cs (verzija 1)

```
using Microsoft.AspNetCore.Mvc;
using MojaApp.API.Data;
using MojaApp.API.Dtos;
using MojaApp.API.Models;

namespace MojaApp.API.Controllers;

[ApiController]
[Route("[controller]")]
public class StudentiController(StudentStorage storage) : ControllerBase
{
    [HttpGet]
    public List<StudentResponse> GetAll()
    {
        return storage.GetAll().Select(Mapiraj).ToList();
    }

    private static StudentResponse Mapiraj(Student s) => new()
    {
        Id = s.Id,
        ImePrezime = $"{s.Ime} {s.Prezime}",
        BrojIndeksa = s.BrojIndeksa,
        DatumRodjenja = s.DatumRodjenja,
    };
}
```

- `StudentiController(StudentStorage storage)` je **primary constructor**: kontroler kroz konstruktor dobija skladište iz DI-ja. Isti stil koristi i starter projekat.
- `[HttpGet]` bez dodatka rute obrađuje `GET /Studenti`.
- `Select(Mapiraj)` svaki entitet pretvara u DTO, pa klijent nikad ne dobija entitet.
- `Mapiraj` je `private`, pa nije akcija: akcije su samo javne metode.

Pokrenite projekat i u Swaggeru isprobajte `GET /Studenti`.

3.7 Akcija GetById

U kontroler, iznad metode `Mapiraj`, dodajte:

```
    [HttpGet("{id:int}")]
    public ActionResult<StudentResponse> GetById(int id)
    {
        Student? student = storage.GetById(id);

        if (student == null)
            return NotFound(); // 404, a ne izuzetak (500)

        return Mapiraj(student);
    }
```

- `"{id:int}"` čita broj iz URL-a u parametar `id`. Ograničenje `:int` znači da `GET /Studenti/abc` uopšte ne ulazi u akciju, nego vraća 404.
- `ActionResult<StudentResponse>` dozvoljava da akcija vrati ili DTO (200), ili rezultat kao `NotFound()` (404).
- Zašto ne `throw new Exception("nema studenta")`? Izuzetak bi postao 500, a klijent ne bi znao da student samo ne postoji. 404 kaže tačno to.

3.8 Request DTO i akcija Dodaj

U folderu `Dtos` napravite klasu za zahtjev:

Dtos/StudentDodajRequest.cs

```
using System.ComponentModel.DataAnnotations;

namespace MojaApp.API.Dtos;

// Ono što klijent šalje. Id i DatumKreiranja postavlja server, pa ih ovdje nema.
public class StudentDodajRequest
{
    [Required, StringLength(50)]
    public string Ime { get; set; } = "";

    [Required, StringLength(50)]
    public string Prezime { get; set; } = "";

    [Required, RegularExpression(@"^IB\d{6}$", ErrorMessage = "Broj indeksa mora biti u formatu
IB123456.")]
    public string BrojIndeksa { get; set; } = "";

    public DateOnly? DatumRodjenja { get; set; }
}
```

U kontroler, iznad metode `Mapiraj`, dodajte akciju:

```
    [HttpPost]
    public ActionResult<StudentResponse> Dodaj(StudentDodajRequest request)
    {
        // [ApiController] sam vraća 400 ako request ne prođe validaciju iz StudentDodajRequest
        Student student = storage.Dodaj(new Student
        {
            Ime = request.Ime,
            Prezime = request.Prezime,
            BrojIndeksa = request.BrojIndeksa,
            DatumRodjenja = request.DatumRodjenja,
        });

        return CreatedAtAction(nameof(GetById), new { id = student.Id }, Mapiraj(student)); // 201 +
Location
    }
```

- Parametar `request` dolazi iz tijela zahtjeva: `[ApiController]` zna da složeni tip čita iz JSON-a.
- Validacija se izvrši prije akcije. Ako broj indeksa nije u formatu IB123456, klijent dobija 400, a akcija se ne izvrši.
- Iz request DTO-a pravimo entitet. `Id` i `DatumKreiranja` postavlja skladište.
- `CreatedAtAction` vraća 201, novog studenta u tijelu i zaglavlje `Location` s adresom novog resursa, npr. `/Studenti/3`.

Odgovor kad validacija ne prođe:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "BrojIndeksa": [ "Broj indeksa mora biti u formatu IB123456." ]
  }
}
```

3.9 Kompletan kontroler

Nakon koraka 3.6–3.8 kontroler treba izgledati ovako:

Controllers/StudentiController.cs

```
using Microsoft.AspNetCore.Mvc;
using MojaApp.API.Data;
using MojaApp.API.Dtos;
using MojaApp.API.Models;

namespace MojaApp.API.Controllers;

[ApiController]
[Route("[controller]")]
public class StudentiController(StudentStorage storage) : ControllerBase
{
    [HttpGet]
    public List<StudentResponse> GetAll()
    {
        return storage.GetAll().Select(Mapiraj).ToList();
    }

    [HttpGet("{id:int}")]
    public ActionResult<StudentResponse> GetById(int id)
    {
        Student? student = storage.GetById(id);

        if (student == null)
            return NotFound(); // 404, a ne izuzetak (500)

        return Mapiraj(student);
    }

    [HttpPost]
    public ActionResult<StudentResponse> Dodaj(StudentDodajRequest request)
    {
        // [ApiController] sam vraća 400 ako request ne prođe validaciju iz StudentDodajRequest
        Student student = storage.Dodaj(new Student
        {
            Ime = request.Ime,
            Prezime = request.Prezime,
            BrojIndeksa = request.BrojIndeksa,
            DatumRodjenja = request.DatumRodjenja,
        });

        return CreatedAtAction(nameof(GetById), new { id = student.Id }, Mapiraj(student)); // 201 +
Location
    }

    private static StudentResponse Mapiraj(Student s) => new()
    {
        Id = s.Id,
        ImePrezime = $"{s.Ime} {s.Prezime}",
        BrojIndeksa = s.BrojIndeksa,
        DatumRodjenja = s.DatumRodjenja,
    };
}
```

3.10 Testiranje

U Swaggeru isprobajte:

Zahtjev	Očekivano
`GET /Studenti`	200, dva studenta
`GET /Studenti/1`	200, Ana Anić
`GET /Studenti/999`	404

Zahtjev	Očekivano
POST /Studenti s ispravnim podacima	201 i zaglavlje Location
POST /Studenti s brojem indeksa 123	400 s porukom o formatu

Iste zahtjeve možete slati iz fajla `MojaApp.API.http`, klikom na „Send request" iznad zahtjeva. Port u prvom redu zamijenite portom iz `Properties/launchSettings.json`, profil `http`:

MojaApp.API.http

```
@MojaApp.API_HostAddress = http://localhost:5183

### Svi studenti
GET {{MojaApp.API_HostAddress}}/studenti
Accept: application/json

### Student po Id-u (200)
GET {{MojaApp.API_HostAddress}}/studenti/1

### Nepostojeći student (404)
GET {{MojaApp.API_HostAddress}}/studenti/999

### Dodavanje (201)
POST {{MojaApp.API_HostAddress}}/studenti
Content-Type: application/json

{
  "ime": "Lejla",
  "prezime": "Lejlić",
  "brojIndeksa": "IB260003",
  "datumRodjenja": "2006-09-01"
}

### Neispravan broj indeksa (400)
POST {{MojaApp.API_HostAddress}}/studenti
Content-Type: application/json

{
  "ime": "Test",
  "prezime": "Test",
  "brojIndeksa": "123"
}
```

Restartujte aplikaciju i ponovo pošaljite GET `/Studenti`: dodani student je nestao, jer su podaci samo u memoriji. Zato u tački 5 dodajemo bazu (EF Core i SQL Server).

4. Zadaci

Svaki zadatak radite kroz issue, vlastitu granu i pull request s redom `Closes #N`, kako je opisano u „V01 - git branch i PR".

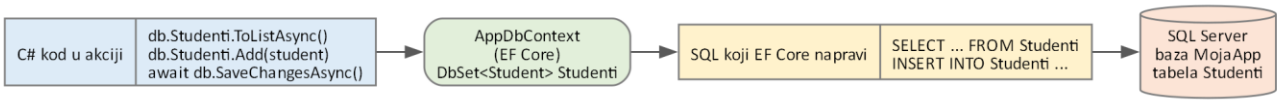
#	Zadatak	Odgovor API-ja	Uputa
1	Brisanje studenta: DELETE `/Studenti/{id}`	204; 404	U skladištu metoda `Obrisi(int id)` koja vraća `bool`. Akcija vraća `IActionResult`: `NoContent()` ili `NotFound()`.
2	Izmjena studenta: PUT `/Studenti/{id}`	204; 404; 400	Novi DTO `StudentUrediRequest` s imenom, prezimenom i datumom rođenja; broj indeksa se ne mijenja. Nađite entitet i promijenite polja.
3	Pretraga: GET `/Studenti?pretraga=ana`	200, filtrirana lista	Parametar `[FromQuery] string? pretraga`. Tražite po imenu, prezimenu i broju indeksa: `Contains(…,`

#	Zadatak	Odgovor API-ja	Uputa
			`StringComparison.OrdinalIgnoreCase`).
4	Jedinstven broj indeksa pri dodavanju	409	Prije dodavanja provjerite postoji li isti broj indeksa. Ako postoji, `return Conflict("…")`.

Kad pređete na bazu (tačka 5), iste zadatke radite nad `AppDbContext`: brisanje je `db.Studenti.Remove(student)`, izmjena je promjena polja na entitetu, a u oba slučaja na kraju ide `await db.SaveChangesAsync(ct)`.

5. Sljedeći zadatak: baza na SQL Serveru

Do sada su studenti bili u listi u memoriji i nestajali su s restartom aplikacije. Sada ih spremamo u bazu na SQL Serveru. Između koda i baze stoji **Entity Framework Core (EF Core)**: vi pišete C#, a EF Core pravi SQL.



- **EF Core** je biblioteka koja klase preslikava u tabele, a C# upite u SQL.
- **DbContext** je most između koda i baze. Pravi se novi za svaki HTTP zahtjev (Scoped).
- **DbSet** je jedna tabela: `DbSet<Student> Studenti` je tabela Studenti.
- **Connection string** je adresa baze: server, naziv baze i prijava.
- **Migracija** je C# fajl koji opisuje kako napraviti ili promijeniti tabele. EF Core je pravi iz vaših klasa.

5.1 NuGet paketi

Desni klik na projekat → Manage NuGet Packages → Browse, pa instalirajte oba paketa, verzija 8.0.x (kao u starter projektu). Isto iz komandne linije:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.21
dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.21
```

5.2 Connection string

U `appsettings.json` dodajte sekciju `ConnectionStrings`; ostatak fajla ostaje isti:

```
"ConnectionStrings": {
  "Main": "Server=localhost,1433;Database=MojaApp;User ID=sa;Password=test;TrustServerCertificate=True"
},
```

SQL Server je podešen isto kao za starter projekat (zadaća iz V01). Bazu MojaApp ne pravite ručno, nego je pravi migracija.

5.3 AppDbContext

U folderu `Data` napravite klasu:

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using MojaApp.API.Models;

namespace MojaApp.API.Data;

// Most između C# koda i baze: svaki DbSet je jedna tabela.
public class AppDbContext(DbContextOptions<AppDbContext> options) : DbContext(options)
{
    public DbSet<Student> Studenti => Set<Student>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Početni podaci: migracija ih upisuje u tabelu.
        modelBuilder.Entity<Student>().HasData(
            new Student { Id = 1, Ime = "Ana", Prezime = "Anić", BrojIndeksa = "IB260001", DatumRodjenja =
new DateOnly(2006, 3, 14), DatumKreiranja = new DateTime(2026, 10, 1) },
            new Student { Id = 2, Ime = "Edin", Prezime = "Edić", BrojIndeksa = "IB260002", DatumKreiranja
= new DateTime(2026, 10, 1) });
    }
}
```

- `DbSet<Student> Studenti` postaje tabela Studenti, s kolonama prema propertijima klase `Student`.
- `HasData` upisuje početne podatke pri migraciji. Datum je fiksan, jer migracija mora svaki put dati isti rezultat.

5.4 Program.cs

Umjesto `AddSingleton<StudentStorage>()` registrujte DbContext, a fajl `Data/StudentStorage.cs` obrišite:

```
using Microsoft.EntityFrameworkCore;   // na vrh fajla

// Baza: connection string "Main" iz appsettings.json.
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("Main")));
```

`AddDbContext` registruje DbContext kao Scoped: svaki zahtjev dobija svoj.

5.5 Kontroler nad bazom

Controllers/StudentiController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MojaApp.API.Data;
using MojaApp.API.Dtos;
using MojaApp.API.Models;

namespace MojaApp.API.Controllers;

[ApiController]
[Route("[controller]")]
public class StudentiController(AppDbContext db) : ControllerBase
{
    [HttpGet]
    public async Task<List<StudentResponse>> GetAll(CancellationToken ct)
    {
        List<Student> studenti = await db.Studenti.ToListAsync(ct); // SELECT
        return studenti.Select(Mapiraj).ToList();
    }

    [HttpGet("{id:int}")]
    public async Task<ActionResult<StudentResponse>> GetById(int id, CancellationToken ct)
    {
        Student? student = await db.Studenti.FirstOrDefaultAsync(x => x.Id == id, ct); // SELECT ... WHERE
Id = @id

        if (student == null)
            return NotFound();

        return Mapiraj(student);
    }

    [HttpPost]
    public async Task<ActionResult<StudentResponse>> Dodaj(StudentDodajRequest request, CancellationToken
ct)
    {
        Student student = new()
        {
            Ime = request.Ime,
            Prezime = request.Prezime,
            BrojIndeksa = request.BrojIndeksa,
            DatumRodjenja = request.DatumRodjenja,
            DatumKreiranja = DateTime.UtcNow,
        };

        db.Studenti.Add(student);
        await db.SaveChangesAsync(ct); // INSERT; baza dodjeljuje Id

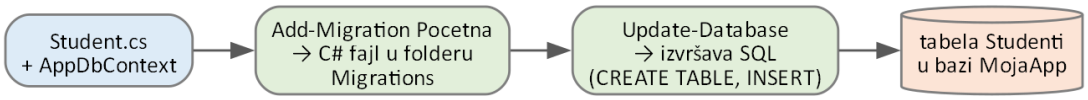
        return CreatedAtAction(nameof(GetById), new { id = student.Id }, Mapiraj(student));
    }

    private static StudentResponse Mapiraj(Student s) => new()
    {
        Id = s.Id,
        ImePrezime = $"{s.Ime} {s.Prezime}",
        BrojIndeksa = s.BrojIndeksa,
        DatumRodjenja = s.DatumRodjenja,
    };
}
```

– Kontroler sada dobija `AppDbContext db` umjesto skladišta.

- `async`, `await` i `Task<...>`: dok baza radi, server ne stoji i može obrađivati druge zahtjeve.
- `ToListAsync` postaje SELECT, a `FirstOrDefaultAsync(x => x.Id == id)` postaje SELECT … WHERE Id = ….
- `Add` samo priprema novi red. Tek `SaveChangesAsync` šalje INSERT u bazu. Baza tada dodjeljuje `Id`, pa ga `CreatedAtAction` već ima.
- `CancellationToken ct`: ako klijent odustane od zahtjeva, prekida se i upit prema bazi.

5.6 Migracija



Visual Studio → Tools → NuGet Package Manager → Package Manager Console:

```
Add-Migration Pocetna
Update-Database
```

Ili iz komandne linije, u folderu projekta (alat `dotnet-ef` se instalira jednom):

```
dotnet tool install --global dotnet-ef --version 8.*
dotnet ef migrations add Pocetna
dotnet ef database update
```

U SSMS-u provjerite: postoji baza MojaApp, a u njoj tabela Studenti s dva reda.

5.7 Testiranje

Ponovite testove iz tačke 3.10. Zatim restartujte aplikaciju i pošaljite `GET /Studenti`: dodani student je i dalje tu, jer je u bazi.

Greška	Šta provjeriti
„A network-related or instance-specific error occurred while establishing a connection to SQL Server”	Radi li SQL Server servis i je li uključen TCP/IP na portu 1433.
„Login failed for user 'sa'”	Je li uključena SQL Server prijava i jesu li korisnik i lozinka kao u connection stringu.
„The term 'Add-Migration' is not recognized”	Je li instaliran paket `Microsoft.EntityFrameworkCore.Tools` i je li u Package Manager Consoleu izabran projekat MojaApp.API.
Migracija javlja „Build failed”	Projekat se mora buildati prije migracije: prvo ispravite greške u kodu.

6. Provjerite da znate odgovoriti

- Koja je razlika između kontrolera i akcije?
- Kako ASP.NET zna da `GET /Studenti/5` ide u `GetById` i da je `id` = 5?
- Zašto akcija `Dodaj` prima `StudentDodajRequest`, a ne `Student`?
- Zašto `StudentResponse` ima `ImePrezime`, a entitet ima `Ime` i `Prezime`?
- Ko vraća 400 kad broj indeksa nije ispravan i da li se akcija `Dodaj` tada izvrši?
- Šta vraća `CreatedAtAction` i šta piše u zaglavlju `Location`?
- Zašto je `StudentStorage` registrovan kao singleton? Šta bi se desilo sa `AddScoped`?
- Zašto `Mapiraj` nije akcija?

- Zašto je DbContext Scoped, a skladište u memoriji je bilo singleton?
- Kada EF Core zaista pošalje INSERT u bazu?